SYSTEMS, DEVOPS & PLATFORM ENGINEERING

Learning Roadmap

A structured, project-driven path toward secure, automated, observable, reliable, and AI-aware infrastructure.

FOCUS	Platform Engineering · DevSecOps · Cloud Security · SRE
STRUCTURE	5 Levels · 25 Modules · 5 Milestone Projects
DOCUMENT	English Edition · Project-Driven Learning Journal

About This Roadmap

ENGLISH EDITION

This document presents a structured, project-driven learning roadmap designed to build a strong technical foundation in systems administration, networking, cybersecurity, DevSecOps, cloud computing, Site Reliability Engineering, platform engineering, and AI systems security. The roadmap combines concise technical scope definitions, security-focused analysis, practical validation work, and five progressively advanced portfolio projects. Each level extends the previous one so that knowledge is demonstrated through systems that can be built, tested, documented, and reviewed.

Roadmap Design Principles

- Build systems progressively, beginning with networking and Linux fundamentals.
- Connect security controls to the systems, applications, and platforms they protect.
- Validate important claims through tests, commands, metrics, logs, and documented evidence.
- Treat architectural limitations and operational trade-offs as part of the engineering work.
- Complete each level with one realistic portfolio project that extends the previous milestone.

Structure at a Glance

Levels	5 progressive learning levels
Modules	25 focused technical modules
Projects	5 connected milestone portfolio projects
Language	English

Contents

Roadmap Overview

Level	Primary Scope	Modules
1	Foundations of Systems & Security	01-03
2	Application Infrastructure, Containers & Infrastructure as Code	04-07
3	Software Engineering, Automation, CI/CD & Observability	08-11
4	Cloud Computing, Reliability & Intelligent Operations	12-15
5	AI Systems Security, MLSecOps & Governance	16-25

The learning path progresses from systems and networking foundations to cloud reliability and AI systems security. Each level closes with one portfolio project that extends the system built in the previous level.

LEVEL 1 | Foundations of Systems & Security

Systems, Networking, Linux and Security Foundations

Build the operational foundation required to understand, administer, troubleshoot, and protect Linux-based infrastructure.

MODULE 01 | Advanced Networking for Platforms

- **Physical & Data Link Layers** - OSI and TCP/IP models, encapsulation, physical signals, Ethernet frame structure, MAC addressing, switching, MTU, interface counters, and Layer 1/2 troubleshooting.
- **ARP & Local Network Dynamics** - ARP behavior, neighbor tables, ARP cache states, gratuitous ARP, proxy ARP, ARP spoofing risks, and Dynamic ARP Inspection.
- **VLAN & VXLAN** - Layer 2 segmentation, access and trunk ports, IEEE 802.1Q tagging, VLAN hopping risks, VXLAN overlay architecture, VTEPs, and VNIs.
- **Network Routing & Subnetting** - IPv4 addressing, CIDR, subnetting, routing tables, default gateways, static and dynamic routing, OSPF/BGP concepts, and IP fragmentation.
- **Transport Layer & Traffic Control** - TCP and UDP, ports and sockets, the TCP three-way handshake, sequence and acknowledgment numbers, flow control, congestion control, retransmission, and connection termination.
- **Application Layer Services** - DNS record types and resolution, DNS spoofing risks, the DHCP DORA process, DHCP Snooping, HTTP/HTTPS, and the TLS handshake.
- **Load Balancing & Proxy Concepts** - Forward and reverse proxies, Layer 4 and Layer 7 load balancing, health checks, session persistence, TLS termination, and TLS passthrough.

MODULE 02 | Enterprise Linux System Administration

- **Linux Architecture, Boot Process & FHS** - The kernel, initramfs, GRUB, systemd, the Linux boot sequence, and the filesystem hierarchy including /etc, /var, /home, /opt, /tmp, /proc, /sys, and /dev.
- **Shell, CLI & Stream Processing** - Standard input and output, pipes, redirection, environment variables, and tools such as grep, awk, sed, cut, sort, uniq, find, and xargs.
- **User & Permission Management** - User and group administration, UID/GID, chmod, chown, umask, POSIX ACLs, sudoers, and least-privilege access.
- **Process & Resource Management** - Process lifecycles, signals, foreground and background jobs, ps, top, htop, kill, file descriptors, memory, and CPU monitoring.
- **systemd, Services & Scheduled Tasks** - Unit files, service dependencies, systemctl, journalctl, restart policies, systemd timers, and cron.
- **Storage, Filesystems & LVM** - Partitioning, block devices, ext4, XFS, mounts, /etc/fstab, inodes, swap, LVM, RAID, and disk-capacity management.
- **Linux Networking & Troubleshooting** - Interface management, routing, DNS resolution, /etc/hosts, systemd-resolved, ip, ss, tcpdump, dig, curl, nc, and network namespaces.
- **Package & Repository Management** - apt, dpkg, dnf, rpm, repository architecture, package signatures, dependency management, and secure update processes.
- **Logging, Time & Operational Maintenance** - /var/log, the systemd journal, rsyslog, log rotation, NTP, chrony, backup, restore validation, and maintenance planning.

MODULE 03 | Security Engineering & Incident Management

- **Security Foundations & Risk Management** - The CIA triad, authentication, authorization, accountability, assets, threats, vulnerabilities, risk, attack surface, and defense in depth.
- **Threat Modeling & Attack Lifecycle** - Trust boundaries, data-flow diagrams, STRIDE, attack trees, the Cyber Kill Chain, MITRE ATT&CK, and the attack lifecycle.
- **Network and Availability Threats** - MITM, spoofing, session hijacking, DNS and ARP attacks, DoS/DDoS, SYN floods, and service-disruption scenarios.
- **Malware & Endpoint Defense** - Worms, Trojans, ransomware, rootkits, botnets, fileless malware, sandboxing, hashes, EDR, and endpoint hardening.
- **Firewalling & Network Defense** - Stateful and stateless filtering, Netfilter, iptables, nftables, connection tracking, NAT, rate limiting, and segmentation.
- **Cryptography, PKI & SSH Security** - Symmetric and asymmetric cryptography, hashing, digital signatures, certificates, SSH key pairs, host keys, and sshd_config hardening.
- **Linux Hardening & Vulnerability Management** - Attack-surface reduction, CIS Benchmarks, CVE/CVSS, patching, service inventory, permissions, PAM, SELinux, and AppArmor fundamentals.
- **System Auditing, Logging & SIEM** - /var/log, the systemd journal, auditd, log integrity, centralized logging, SIEM, detection rules, and security-event analysis.
- **Log-Based Abuse Mitigation** - Fail2Ban behavior, authentication-log analysis, temporary blocking, rate limiting, and the operational limitations of log-based controls.
- **Incident Response Fundamentals** - Preparation, detection, triage, containment, eradication, recovery, evidence preservation, communication, and post-incident review.

LEVEL 2 | Application Infrastructure, Containers & Infrastructure as Code

Application, Data, Container and Infrastructure Automation

Learn how web applications, APIs, databases, containers, Kubernetes, and Infrastructure as Code work together as an application platform.

MODULE 04 | Web, API & Database Foundations

- **Web Architecture Fundamentals** - The client-server model, request-response flow, and stateless versus stateful application design.
- **HTTP & HTTPS** - HTTP methods, status codes, headers, content types, caching, HTTP/1.1, HTTP/2, HTTP/3, and HTTPS.
- **TLS & Certificate Validation** - The TLS handshake, X.509 certificates, Certificate Authorities, SNI, certificate chains, validation, and renewal processes.
- **Cookies, Sessions & Authentication** - Cookies, session management, authentication state, secure cookie attributes, and session security.
- **REST API Fundamentals** - Resources, endpoints, HTTP methods, idempotency, pagination, versioning, rate limiting, and API error handling.
- **Tokens, JWT & OAuth 2.0** - Access and refresh tokens, JWT structure, OAuth 2.0 flows, and OpenID Connect fundamentals.
- **Browser & API Security** - CORS, CSRF, XSS, Host-header risks, input validation, and the distinction between authentication and authorization.
- **Relational Database Fundamentals** - PostgreSQL and MySQL architecture, schemas, tables, primary and foreign keys, and normalization.
- **Transactions & Concurrency** - ACID, transaction isolation levels, locks, deadlocks, and consistency models.
- **Indexes & Query Performance** - Index structures, query plans, full-table scans, latency, and fundamental query optimization.
- **Database Availability & Protection** - Replication, backup, restore testing, encryption, access control, and credential management.
- **Caching & In-Memory Data Stores** - Redis fundamentals, cache-aside patterns, expiration, persistence, and cache-invalidation challenges.

MODULE 05 | Container Technologies — Docker Deep Dive

- **Containerization vs Virtualization** - Architectural differences between hypervisor-based virtual machines and operating-system-level containers.
- **Linux Kernel Namespaces & Cgroups** - The Linux kernel mechanisms that provide container isolation and resource control.
- **Container Runtime Internals** - Docker Engine, containerd, runc, OCI specifications, and the Kubernetes Container Runtime Interface.
- **Docker Core** - The Docker CLI, image and container lifecycles, Dockerfile practices, layers, build cache, and multi-stage builds.
- **Docker Storage** - Writable layers, volumes, bind mounts, tmpfs, UID/GID concerns, and data persistence.
- **Docker Networking** - Network namespaces, veth pairs, bridge networks, port publishing, NAT, container DNS, and service discovery.
- **Docker Compose** - Defining repeatable multi-container environments for local development, testing, and validation.
- **Registries & Image Lifecycle** - Docker Hub, GHCR, Amazon ECR, tags, digests, retention, signing, and image provenance.
- **Container Security & Image Hardening** - Non-root containers, reduced capabilities, read-only filesystems, seccomp, minimal images, and Trivy/Grype scanning.
- **Docker Operations & Troubleshooting** - Logs, inspect, events, health checks, networking, storage, OOM behavior, and safe cleanup practices.

MODULE 06 | Kubernetes Orchestration & Platform Fundamentals

- **Kubernetes Architecture** - The API Server, etcd, Scheduler, controller managers, kubelet, kube-proxy, and container-runtime responsibilities.
- **Kubernetes Core Objects** - Pods, ReplicaSets, Deployments, StatefulSets, DaemonSets, Jobs, and CronJobs.
- **Scheduling & Resources** - Requests, limits, QoS classes, affinity, anti-affinity, taints, tolerations, and pod placement.
- **Services & Service Discovery** - ClusterIP, NodePort, LoadBalancer Services, EndpointSlices, kube-proxy, and CoreDNS.
- **Configuration & Secrets** - ConfigMaps, Secrets, environment variables, mounted volumes, and the security limitations of Kubernetes Secrets.
- **Kubernetes Storage** - PersistentVolumes, PersistentVolumeClaims, StorageClasses, dynamic provisioning, access modes, and backup considerations.
- **Kubernetes Security & RBAC** - ServiceAccounts, Roles, ClusterRoles, bindings, security contexts, Pod Security Standards, and least privilege.
- **Traffic Management** - Ingress controllers, Gateway API, NetworkPolicy, service meshes, mTLS, retries, and circuit breaking.
- **Workload Health & Recovery** - Startup, readiness, and liveness probes; restart behavior; and how controllers maintain desired state.
- **Scaling & Reliability** - HPA, VPA concepts, Cluster Autoscaler, PodDisruptionBudget, graceful shutdown, and controlled rollout strategies.
- **Cluster Lifecycle & Packaging** - kubeadm, managed Kubernetes, upgrades, Helm, Kustomize, certificates, and etcd backup fundamentals.
- **Kubernetes Observability & Troubleshooting** - Events, logs, Metrics Server, Prometheus, CrashLoopBackOff, ImagePullBackOff, DNS, and volume failures.
- **GitOps & Declarative Delivery** - Declarative management of Kubernetes resources with Argo CD or Flux.

MODULE 07 | Infrastructure as Code & Configuration Management

- **Infrastructure as Code Fundamentals** - Declarative and imperative approaches, desired state, idempotency, drift, convergence, and repeatability.
- **Terraform Language & Workflow** - Providers, resources, data sources, HCL, dependencies, init, plan, apply, and destroy.
- **Terraform Variables & Data Modeling** - Variables, outputs, locals, lists, maps, objects, conditionals, count, and for_each.
- **Terraform Modules** - Root and child modules, reusable architecture, module boundaries, version pinning, and environment separation.
- **Terraform State Security** - terraform.tfstate protection, Amazon S3 remote state, native S3 locking, encryption, versioning, and restricted IAM access.
- **Legacy State Locking Considerations** - Evaluating DynamoDB-based locking for compatibility and migration from legacy Terraform environments.
- **Terraform Lifecycle & Testing** - Lifecycle rules, import, moved blocks, static analysis, Policy as Code, security scanning, and destructive-change protection.
- **Ansible Fundamentals** - Agentless architecture, SSH connectivity, inventories, modules, facts, ad-hoc commands, and privilege escalation.
- **Ansible Playbooks & Roles** - Playbooks, handlers, templates, variables, loops, conditionals, roles, and idempotent service management.
- **Ansible Vault & Operational Safety** - Encrypting sensitive values, secret injection, least privilege, serial changes, and rollback limitations.
- **Git & Version Control Integration** - Commit discipline, pull requests, protected branches, code review, GitFlow, and trunk-based development.
- **Infrastructure Validation** - Formatting, validation, linting, policy checks, controlled plan/apply workflows, and CI/CD integration.
- **Terraform & Ansible Integration** - Separating provisioning from configuration management, using dynamic inventory, and validating the resulting environment.

LEVEL 3 | Software Engineering, Automation, CI/CD & Observability

Software Delivery, Automation and Observability

Develop maintainable infrastructure software, automate delivery workflows, and observe systems through metrics, logs, traces, and actionable alerts.

MODULE 08 | CI/CD Pipelines & GitOps

- **CI/CD Core Concepts** - The differences among Continuous Integration, Continuous Delivery, and Continuous Deployment.
- **Git Workflows & Pipeline Triggers** - Branching strategies, pull requests, protected branches, tags, releases, and event-based triggers.
- **Pipeline Platforms** - Workflows, jobs, runners, stages, artifacts, and environment management with GitHub Actions or GitLab CI.
- **Build, Test & Artifact Pipelines** - Unit, integration, and end-to-end tests; caching; matrices; reports; and reproducible builds.
- **Container Build & Delivery** - Docker/BuildKit, image tagging, commit-SHA traceability, registry authentication, scanning, signing, and provenance.
- **DevSecOps & Security Gates** - SonarQube, Gitleaks, TruffleHog, Trivy, SAST, SCA, IaC scanning, and controlled policy enforcement.
- **Artifact Management** - Registry and artifact-repository solutions including Docker Hub, GHCR, Amazon ECR, and JFrog Artifactory.
- **Secure Pipeline Authentication** - Using OpenID Connect and short-lived identities instead of long-lived cloud credentials.
- **Software Supply-Chain Security** - SBOMs, dependency trust, signing, Sigstore, SLSA, and artifact-integrity controls.
- **Deployment Strategies** - Rolling updates, blue/green, canary, feature flags, progressive delivery, and rollback.
- **GitOps Delivery** - Separating application code from deployment state through declarative manifests, reconciliation, and drift correction.
- **Pipeline Observability & Troubleshooting** - Pipeline metrics, runner utilization, flaky tests, failures, retries, concurrency, and runbooks.

MODULE 09 | Systems Automation & Scripting — Bash & Python

- **Bash Scripting Fundamentals** - Variables, conditionals, loops, functions, arguments, arrays, exit codes, and debugging.
- **Shell Safety Practices** - Quoting, input validation, temporary files, cleanup traps, and context-aware use of set -Eeuo pipefail.
- **Regular Expressions & Text Processing** - Regular expressions and tools including grep, sed, awk, cut, sort, uniq, tr, find, xargs, jq, and yq.
- **System Administration Automation** - Backup, restore validation, disk monitoring, log rotation, service-health checks, and reporting.
- **Python Fundamentals for Operations** - Data types, functions, exceptions, virtual environments, type hints, logging, and configuration.
- **Python for Filesystems & Processes** - os, pathlib, sys, shutil, subprocess, signals, permissions, and safe command execution.
- **API Automation** - Authentication, pagination, timeouts, retries, exponential backoff, rate limiting, and schema validation.
- **Network & Security Automation** - DNS checks, TCP/UDP connectivity, certificate inspection, log parsing, firewall validation, and alert enrichment.
- **Secure Automation** - Secret management, least privilege, auditability, idempotency, and destructive-action safeguards.
- **Automation Quality & Packaging** - Unit tests, mocking, linting, type checking, dependency pinning, CLI design, and CI integration.

MODULE 10 | Software Engineering for Infrastructure & Platform Teams

- **Software Engineering Fundamentals** - Requirements, maintainability, readability, testability, and technical-debt management.
- **Clean Code Principles** - Naming, small functions, separation of concerns, error handling, and understandable abstractions.
- **SOLID Principles** - Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.
- **Design Patterns** - Factory, Strategy, Adapter, Observer, Dependency Injection, and infrastructure-automation examples.
- **Testing Strategies** - Unit, integration, end-to-end, contract, and smoke tests; mocks, stubs, and test doubles.
- **Refactoring & Code Review** - Safe refactoring, code smells, pull-request review, and incremental improvement.
- **Application Architecture** - Monoliths, modular monoliths, microservices, event-driven architecture, and architectural trade-offs.
- **Distributed-System Fundamentals** - Partial failures, timeouts, retries, idempotency, consistency, and eventual consistency.
- **API & Service Design** - Versioning, backward compatibility, error contracts, rate limits, and graceful degradation.
- **Configuration & Dependency Management** - Configuration separation, feature flags, dependency pinning, and environment consistency.
- **Secure Software Development** - Input validation, secret handling, dependency risk, threat modeling, and secure defaults.
- **Engineering Documentation** - Architecture Decision Records, diagrams, runbooks, API documentation, and operational handover.

MODULE 11 | Observability, Logging & Monitoring

- **Observability Fundamentals** - The roles of metrics, logs, traces, events, and profiles and how they work together.
- **Monitoring Approaches** - White-box, black-box, synthetic monitoring, health checks, and baselines.
- **The Four Golden Signals** - Latency, traffic, errors, and saturation.
- **RED & USE Methods** - Observation methods for request-driven services and infrastructure resources.
- **Metrics & Prometheus** - Prometheus architecture, service discovery, scraping, exporters, PromQL, recording rules, and alerting rules.
- **Grafana & Dashboard Engineering** - Dashboards, panels, variables, annotations, thresholds, provisioning, and dashboard anti-patterns.
- **Centralized Logging** - Log aggregation, parsing, retention, and analysis with Elasticsearch-based platforms or Grafana Loki.
- **Distributed Tracing** - OpenTelemetry instrumentation and Collector pipelines with Jaeger, Grafana Tempo, or compatible backends.
- **Alerting & Notification Engineering** - Alertmanager routing, grouping, deduplication, inhibition, silences, and escalation.
- **SLI, SLO & Reliability Monitoring** - Availability, latency, error budgets, burn rate, and SLO-based alerting.
- **Platform & Kubernetes Observability** - Telemetry for hosts, containers, nodes, pods, workloads, and the control plane.
- **Security Observability** - Authentication, authorization, audit, network, DNS, firewall, and cloud-security telemetry.
- **Operational Reliability** - Actionable alerts, alert fatigue, missing telemetry, runbooks, and incident-response integration.
- **Observability Troubleshooting** - Scrape failures, missing logs, trace gaps, cardinality explosions, storage issues, and query-performance problems.

LEVEL 4 | Cloud Computing, Reliability & Intelligent Operations

Cloud, Reliability, Performance and Intelligent Operations

Design measurable, resilient, scalable, recoverable, and cost-aware cloud systems using AWS, SRE, performance engineering, and AI-assisted operations.

MODULE 12 | Cloud Computing — AWS Focus

- **Cloud Computing Foundations** - On-premises versus cloud, IaaS, PaaS, SaaS, elasticity, scalability, and the Shared Responsibility Model.
- **AWS Global Infrastructure** - Regions, Availability Zones, edge locations, and regional versus global services.
- **Identity & Access Management** - IAM users, groups, roles, policies, trust policies, STS, and the Principle of Least Privilege.
- **AWS Networking & VPC Architecture** - VPCs, CIDR planning, public and private subnets, route tables, Internet Gateways, NAT Gateways, and VPC endpoints.
- **Network Security** - Security Groups, Network ACLs, VPC Flow Logs, segmentation, and egress-control approaches.
- **Compute Services** - Amazon EC2, AMIs, launch templates, user data, the metadata service, Auto Scaling Groups, and instance lifecycle.
- **Load Balancing** - Application and Network Load Balancers, listeners, target groups, health checks, and TLS.
- **Storage Services** - Amazon S3, EBS, and EFS use cases and their availability, durability, performance, and cost trade-offs.
- **Managed Databases** - The general architectures of Amazon RDS, Aurora, DynamoDB, and ElastiCache.
- **Serverless & Event-Driven Architecture** - AWS Lambda, SQS, SNS, EventBridge, retries, dead-letter queues, and idempotency.
- **Secrets & Key Management** - AWS Secrets Manager, Systems Manager Parameter Store, AWS KMS, and envelope encryption.
- **Cloud Security Services** - CloudTrail, Config, GuardDuty, Security Hub, Inspector, WAF, Shield, and Macie fundamentals.
- **Monitoring & Auditing** - CloudWatch metrics, logs, alarms, dashboards, CloudTrail events, and centralized logging.
- **High Availability & Disaster Recovery on AWS** - Multi-AZ, Multi-Region, backups, snapshots, replicas, Route 53 failover, RTO, and RPO.
- **Governance & Cost Management** - AWS Organizations, OUs, SCPs, tagging, budgets, Cost Explorer, quotas, and guardrails.
- **AWS & Infrastructure as Code Integration** - Terraform, remote state, OIDC identities, environment promotion, and drift detection.

MODULE 13 | Site Reliability Engineering & High Availability

- **SRE Foundations & Service Ownership** - The relationship between SRE and DevOps, service boundaries, ownership, operational readiness, and reliability as a product feature.
- **SLI, SLO, SLA & Error Budgets** - Service indicators, objectives, external commitments, measurement windows, and burn rate.
- **Toil Reduction & Reliability Automation** - Repetitive operational work, safe automation, self-service, guardrails, and human-in-the-loop controls.
- **Monitoring, Alerting & On-Call** - Golden Signals, actionable alerts, severity, routing, escalation, handoffs, and on-call health.
- **Incident Management** - Incident detection, declaration, triage, Incident Commander responsibilities, stabilization, communication, and recovery.
- **Blameless Postmortems** - Timelines, impact, contributing factors, root-cause analysis, corrective actions, and learning culture.
- **Resilience Patterns** - Timeouts, retries, exponential backoff, jitter, circuit breakers, bulkheads, rate limiting, and load shedding.
- **High Availability Architectures** - Redundancy, single points of failure, active-passive and active-active designs, quorum, replication, and DNS failover.
- **Disaster Recovery & Business Continuity** - RTO, RPO, backup and restore, Pilot Light, Warm Standby, Active-Active, and recovery testing.
- **Chaos Engineering** - Authorized, controlled, hypothesis-driven failure experiments with abort conditions and blast-radius management.
- **Release Engineering & Change Reliability** - Rolling, blue/green, canary, feature flags, migration safety, and rollback.
- **Operational Readiness** - Runbooks, dashboards, alerts, deployment validation, capacity ownership, and recovery plans.

MODULE 14 | Performance Engineering & Capacity Analysis

- **Performance Engineering Fundamentals** - Latency, throughput, concurrency, utilization, saturation, and bottlenecks.
- **CPU Performance Analysis** - CPU utilization, load average, run queues, context switches, user/system time, and CPU profiling.
- **Memory Performance Analysis** - Memory allocation, cache, swap, page faults, memory pressure, leaks, and OOM behavior.
- **Storage & I/O Performance** - IOPS, throughput, latency, queue depth, filesystem overhead, and disk saturation.
- **Network Performance Analysis** - Bandwidth, packet loss, jitter, retransmissions, connection limits, and socket queues.
- **Application Profiling** - Hot paths, function-level profiling, memory allocation, lock contention, and flame graphs.
- **Database Performance** - Query latency, indexes, execution plans, connection pools, locks, and slow-query analysis.
- **Load, Stress & Soak Testing** - Normal load, peak load, breaking points, long-duration tests, and test-environment limitations.
- **Benchmarking Methodology** - Baselines, controlled experiments, repeatability, warm-up effects, and misleading benchmark risks.
- **Capacity Planning** - Growth forecasting, resource headroom, seasonality, quotas, and scaling decisions.
- **Backpressure & Queue Management** - Queue depth, concurrency limits, rate limiting, and overload behavior.
- **Performance Troubleshooting** - Layered bottleneck analysis using metrics, logs, traces, and profiles.
- **Performance vs Cost Trade-offs** - Vertical and horizontal scaling, overprovisioning, efficiency, and cloud-cost impact.

MODULE 15 | AI-Assisted Engineering & Operational Intelligence

- **AI-Assisted Engineering Fundamentals** - Appropriate uses of AI tools in software, systems, cloud, and security engineering.
- **Prompt Design for Engineers** - Clearly defining requirements, context, constraints, validation criteria, and expected output format.
- **AI-Assisted Code Generation** - Controlled generation of Bash, Python, Terraform, Ansible, and Kubernetes manifest drafts.
- **AI-Assisted Code Review** - Reviewing logic errors, security risks, maintainability issues, race conditions, and missing error handling.
- **Infrastructure Review** - AI-assisted review of Terraform plans, IAM policies, Kubernetes manifests, and network rules.
- **Log & Incident Analysis** - Log summarization, timeline creation, correlation, hypothesis generation, and incident-triage assistance.
- **AI-Assisted Troubleshooting** - Prioritizing likely causes from error messages, metrics, traces, and system outputs.
- **Security Analysis & Threat Hunting** - Detection-rule drafts, IOC enrichment, suspicious-pattern analysis, and false-positive assessment.
- **Documentation & Knowledge Management** - Generating and maintaining READMEs, runbooks, postmortems, architecture documents, and technical summaries.
- **Output Verification** - Independently validating AI-generated commands, code, policies, and technical claims.
- **Hallucination & Context Risks** - Risks created by fabricated commands, outdated information, incorrect assumptions, and missing context.
- **Sensitive Data Protection** - Risks of exposing credentials, logs, source code, personal data, and company information to AI tools.
- **Human Approval & Safety Boundaries** - Human approval for production changes, destructive commands, security decisions, and automated actions.
- **AI-Augmented Operations** - Alert enrichment, anomaly summarization, runbook suggestions, and controlled remediation.
- **Responsible AI Use** - Privacy, auditability, intellectual property, bias, accountability, and organizational-policy requirements.

LEVEL 5 | AI Systems Security, MLSecOps & Governance

AI Systems Security, Secure Model Lifecycle and Governance

Study the architecture and threat surface of AI systems, then secure, test, monitor, and govern LLM, RAG, agentic, and model-serving applications.

MODULE 16 | AI, Machine Learning & LLM Foundations for Security Engineers

- **AI & Machine Learning Foundations** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI & Machine Learning Foundations.
- **AI System Lifecycle** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI System Lifecycle.
- **Model Architecture Fundamentals** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Architecture Fundamentals.
- **Large Language Model Fundamentals** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Large Language Model Fundamentals.
- **Embeddings & Semantic Search** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Embeddings & Semantic Search.
- **Retrieval-Augmented Generation** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Retrieval-Augmented Generation.
- **Vector Databases** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Vector Databases.
- **AI Agents & Tool Use** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI Agents & Tool Use.
- **Model Context Protocol Fundamentals** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Context Protocol Fundamentals.
- **Training, Fine-Tuning & Inference** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Training, Fine-Tuning & Inference.
- **AI Security Terminology** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Security Terminology.

MODULE 17 | AI Threat Modeling & Security Architecture

- **AI System Asset Identification** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI System Asset Identification.
- **AI Data-Flow Diagrams** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI Data-Flow Diagrams.
- **Trust Boundaries** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Trust Boundaries.
- **AI Attack-Surface Analysis** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Attack-Surface Analysis.
- **Threat Actors & Capabilities** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Threat Actors & Capabilities.
- **AI Misuse & Abuse Cases** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI Misuse & Abuse Cases.
- **Threat Modeling Methods** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Threat Modeling Methods.
- **OWASP LLM & Generative AI Risks** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for OWASP LLM & Generative AI Risks.
- **MITRE ATLAS** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for MITRE ATLAS.
- **NIST AI Risk Management Framework** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for NIST AI Risk Management Framework.
- **Security Requirements Engineering** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Security Requirements Engineering.
- **AI Security Architecture Patterns** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Security Architecture Patterns.
- **AI Risk Register** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Risk Register.

MODULE 18 | LLM, RAG & Agentic Application Security

- **Direct Prompt Injection** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Direct Prompt Injection.
- **Indirect Prompt Injection** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Indirect Prompt Injection.
- **System-Prompt Extraction** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for System-Prompt Extraction.
- **Jailbreak Attacks** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Jailbreak Attacks.
- **Sensitive Information Disclosure** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Sensitive Information Disclosure.
- **Insecure Output Handling** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Insecure Output Handling.
- **Excessive Agency** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Excessive Agency.
- **Agent Identity & Authorization** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Agent Identity & Authorization.
- **Tool-Calling Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Tool-Calling Security.
- **MCP Server & Tool Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for MCP Server & Tool Security.
- **Agent Memory Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Agent Memory Security.
- **Context Poisoning** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Context Poisoning.
- **RAG Poisoning** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for RAG Poisoning.
- **Retrieval Authorization** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Retrieval Authorization.
- **Vector Database Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Vector Database Security.
- **Cross-Tenant Data Leakage** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Cross-Tenant Data Leakage.
- **SSRF & External-Resource Abuse** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for SSRF & External-Resource Abuse.
- **Code-Execution & Sandbox Risks** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Code-Execution & Sandbox Risks.
- **Output Validation & Policy Enforcement** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Output Validation & Policy Enforcement.
- **Human-in-the-Loop Controls** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Human-in-the-Loop Controls.
- **Rate Limiting & Cost Controls** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Rate Limiting & Cost Controls.

MODULE 18 | LLM, RAG & Agentic Application Security

- **Secure Conversation Design** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Secure Conversation Design.

MODULE 19 | Adversarial Machine Learning & Model Security

- **Adversarial ML Foundations** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Adversarial ML Foundations.
- **Evasion Attacks** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Evasion Attacks.
- **Adversarial Examples** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Adversarial Examples.
- **Training-Data Poisoning** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Training-Data Poisoning.
- **Backdoor & Trojan Attacks** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Backdoor & Trojan Attacks.
- **Fine-Tuning Attacks** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Fine-Tuning Attacks.
- **Model Extraction** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Extraction.
- **Model Inversion** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Inversion.
- **Membership Inference** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Membership Inference.
- **Model Fingerprinting** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Fingerprinting.
- **Model Theft** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Theft.
- **Privacy Attacks** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Privacy Attacks.
- **Model Integrity Verification** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Integrity Verification.
- **Robustness Evaluation** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Robustness Evaluation.
- **Mitigation Limitations** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Mitigation Limitations.

MODULE 20 | AI Data Security, Privacy & Knowledge Protection

- **AI Data Classification** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI Data Classification.
- **Dataset Inventory & Ownership** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Dataset Inventory & Ownership.
- **Data Lineage & Provenance** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Data Lineage & Provenance.
- **Data Quality & Integrity** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Data Quality & Integrity.
- **Training-Data Access Control** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Training-Data Access Control.
- **Secrets & Credential Detection** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Secrets & Credential Detection.
- **Personal-Data Protection** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Personal-Data Protection.
- **Encryption** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Encryption.
- **RAG Knowledge-Base Protection** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for RAG Knowledge-Base Protection.
- **Prompt & Response Privacy** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Prompt & Response Privacy.
- **Logging & Retention Controls** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Logging & Retention Controls.
- **Tenant Isolation** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Tenant Isolation.
- **Data-Poisoning Detection** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Data-Poisoning Detection.
- **Data Usage Rights** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Data Usage Rights.
- **Privacy-Enhancing Technologies** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Privacy-Enhancing Technologies.
- **Secure Data Deletion** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Secure Data Deletion.

MODULE 21 | Secure AI Infrastructure & Model Serving

- **AI Infrastructure Architecture** - Architecture, configuration, lifecycle management, security controls, operational validation, and troubleshooting for AI Infrastructure Architecture.
- **GPU & Accelerator Fundamentals** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for GPU & Accelerator Fundamentals.
- **GPU Workload Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for GPU Workload Security.
- **Secure Model Serving** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Secure Model Serving.
- **Containerized AI Workloads** - Key metrics, diagnostic methods, bottleneck analysis, tuning considerations, and practical validation for Containerized AI Workloads.
- **Kubernetes for AI Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Kubernetes for AI Security.
- **Model-Serving Platforms** - Architecture, configuration, lifecycle management, security controls, operational validation, and troubleshooting for Model-Serving Platforms.
- **Identity & Workload Authentication** - Key metrics, diagnostic methods, bottleneck analysis, tuning considerations, and practical validation for Identity & Workload Authentication.
- **Secrets & Key Management** - AWS Secrets Manager, Systems Manager Parameter Store, AWS KMS, and envelope encryption.
- **Network Segmentation** - Key metrics, diagnostic methods, bottleneck analysis, tuning considerations, and practical validation for Network Segmentation.
- **API Gateway Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for API Gateway Security.
- **Service-to-Service Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Service-to-Service Security.
- **Multi-Tenancy Isolation** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Multi-Tenancy Isolation.
- **Model Registry Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Model Registry Security.
- **Artifact Storage Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Artifact Storage Security.
- **Secure Sandboxing** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Secure Sandboxing.
- **Availability & Resource Exhaustion** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Availability & Resource Exhaustion.
- **Denial-of-Service & Denial-of-Wallet** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Denial-of-Service & Denial-of-Wallet.
- **AI Infrastructure Observability** - Architecture, configuration, lifecycle management, security controls, operational validation, and troubleshooting for AI Infrastructure Observability.
- **Backup, Recovery & Resilience** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Backup, Recovery & Resilience.
- **Cloud AI Shared Responsibility** - Architecture, configuration, lifecycle management, security controls, operational validation, and troubleshooting for Cloud AI Shared Responsibility.

MODULE 22 | MLSecOps, AI Supply Chain & Secure AI Development Lifecycle

- **MLSecOps Foundations** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for MLSecOps Foundations.
- **Secure AI SDLC** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Secure AI SDLC.
- **AI Pipeline Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Pipeline Security.
- **Secure Development Environments** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Secure Development Environments.
- **Dataset Supply-Chain Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Dataset Supply-Chain Security.
- **Model Supply-Chain Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Model Supply-Chain Security.
- **Third-Party Model Risk** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Third-Party Model Risk.
- **Dependency Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Dependency Security.
- **Unsafe Model Serialization** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Unsafe Model Serialization.
- **Artifact Signing & Verification** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Artifact Signing & Verification.
- **AI Bill of Materials** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI Bill of Materials.
- **Reproducibility** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Reproducibility.
- **Secure CI/CD for AI** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Secure CI/CD for AI.
- **Security Evaluation Gates** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Security Evaluation Gates.
- **Model Registry Promotion** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Registry Promotion.
- **Canary & Shadow Deployment** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Canary & Shadow Deployment.
- **Rollback & Model Revocation** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Rollback & Model Revocation.
- **Drift & Unauthorized Change Detection** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Drift & Unauthorized Change Detection.
- **Vulnerability & Patch Management** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Vulnerability & Patch Management.
- **Vendor & Acquisition Security** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Vendor & Acquisition Security.

MODULE 23 | AI Security Testing, Evaluation & Red Teaming

- **AI Security Test Planning** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Security Test Planning.
- **Prompt Fuzzing** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Prompt Fuzzing.
- **Jailbreak Testing** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Jailbreak Testing.
- **Prompt-Injection Testing** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Prompt-Injection Testing.
- **RAG Security Testing** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for RAG Security Testing.
- **Agentic Security Testing** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Agentic Security Testing.
- **MCP Security Testing** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for MCP Security Testing.
- **Model Extraction Testing** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Extraction Testing.
- **Privacy Leakage Testing** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Privacy Leakage Testing.
- **Adversarial Input Testing** - Design principles, implementation patterns, security considerations, testing, maintainability, and operational validation for Adversarial Input Testing.
- **Multimodal Security Testing** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Multimodal Security Testing.
- **Tool & Output Exploitation** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Tool & Output Exploitation.
- **Abuse & Cost Testing** - Design principles, implementation patterns, security considerations, testing, maintainability, and operational validation for Abuse & Cost Testing.
- **Automated AI Evaluations** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Automated AI Evaluations.
- **Human Red Teaming** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Human Red Teaming.
- **Security vs Safety Evaluation** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Security vs Safety Evaluation.
- **Findings & Severity Assessment** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Findings & Severity Assessment.
- **Remediation Verification** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Remediation Verification.
- **Responsible Testing Boundaries** - Design principles, implementation patterns, security considerations, testing, maintainability, and operational validation for Responsible Testing Boundaries.

MODULE 24 | AI Security Monitoring, Threat Detection & Incident Response

- **AI Security Telemetry** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Security Telemetry.
- **Secure AI Logging** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Secure AI Logging.
- **Prompt & Response Monitoring** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Prompt & Response Monitoring.
- **Agent Activity Monitoring** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Agent Activity Monitoring.
- **RAG Monitoring** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for RAG Monitoring.
- **Model-Behavior Monitoring** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model-Behavior Monitoring.
- **Abuse Detection** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Abuse Detection.
- **Drift Monitoring** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Drift Monitoring.
- **AI Security Metrics** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Security Metrics.
- **SIEM Integration** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for SIEM Integration.
- **Detection Engineering** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Detection Engineering.
- **Threat Hunting with MITRE ATLAS** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Threat Hunting with MITRE ATLAS.
- **AI Incident Classification** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Incident Classification.
- **AI Incident Triage** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Incident Triage.
- **Containment Strategies** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Containment Strategies.
- **Eradication & Recovery** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Eradication & Recovery.
- **Model Rollback & Revocation** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Rollback & Revocation.
- **Credential & Secret Rotation** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Credential & Secret Rotation.
- **AI Forensics & Evidence Preservation** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI Forensics & Evidence Preservation.
- **Communication & Reporting** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Communication & Reporting.
- **Post-Incident Review** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for Post-Incident Review.

MODULE 25 | AI Governance, Assurance, Standards & Compliance

- **AI Governance Foundations** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Governance Foundations.
- **NIST AI RMF** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for NIST AI RMF.
- **NIST Generative AI Profile** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for NIST Generative AI Profile.
- **NIST Secure AI Development Practices** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for NIST Secure AI Development Practices.
- **ISO/IEC 42001** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for ISO/IEC 42001.
- **ISO/IEC 23894** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for ISO/IEC 23894.
- **EU AI Act** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for EU AI Act.
- **CSA AI Controls Matrix** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for CSA AI Controls Matrix.
- **OWASP & MITRE Mapping** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for OWASP & MITRE Mapping.
- **AI System Inventory** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI System Inventory.
- **AI Risk Classification** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Risk Classification.
- **AI Impact Assessments** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for AI Impact Assessments.
- **Model Cards & System Cards** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Model Cards & System Cards.
- **Transparency & Explainability** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Transparency & Explainability.
- **Human Oversight** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Human Oversight.
- **Accountability & Auditability** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Accountability & Auditability.
- **Third-Party Assurance** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Third-Party Assurance.
- **Acceptable AI Use Policies** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Acceptable AI Use Policies.
- **AI Exception & Risk Acceptance** - Threat scenarios, defensive controls, architecture decisions, validation methods, and operational limitations for AI Exception & Risk Acceptance.
- **Continuous Compliance** - Core concepts, architecture, operational behavior, security implications, practical validation, and troubleshooting for Continuous Compliance.
- **Responsible AI Principles** - Architecture, trust boundaries, threat scenarios, security controls, evaluation methods, and governance considerations for Responsible AI Principles.

Level-Based Milestone Portfolio Projects

Each level concludes with one substantial portfolio project that combines the knowledge acquired up to that point.

The projects are deliberately connected. Every new milestone extends the system created in the previous level, producing a portfolio whose growth in security, automation, cloud architecture, observability, reliability, and AI security can be demonstrated clearly.

The project approach is consistent throughout the roadmap:

1. Build a working and understandable minimum version first.
2. Add security and operational controls incrementally.
3. Validate every important control with commands, tests, screenshots, metrics, or sanitized logs.
4. State architectural and operational limitations explicitly.
5. Do not describe a system as production-ready unless that claim has been demonstrated.
6. Publish each project with clear English technical documentation on GitHub.

Level	Milestone Project	Primary Outcome
1	Hardened Linux Server and Security Monitoring Lab	Hardened and auditable Linux server
2	Secure Containerized Application Platform with Infrastructure as Code	Repeatable container and Kubernetes application platform
3	Secure CI/CD, GitOps and Observability Platform	Secure CI/CD, GitOps and observability workflow
4	Resilient AWS Service Platform and SRE Validation	Resilient AWS service with SRE validation
5	Secure RAG Security Operations Assistant	Secure RAG application with AI security controls

Project 1 - Hardened Linux Server and Security Monitoring Lab

> Timeline: After completing Level 1.

Objective

Combine networking, Linux administration, and foundational security knowledge to deploy, harden, monitor, and investigate an internet-accessible Ubuntu server.

What Will Be Built?

A single Ubuntu Server instance will run on Amazon EC2 and expose only a small Nginx status page. The project is not centered on web development; it demonstrates operating-system security, network access control, logging, auditing, and recovery procedures.

Implementation Work

1. Inspect interfaces, IP addressing, routing tables, DNS, listening ports, and active connections.
2. Configure users, groups, sudo privileges, and file permissions according to least privilege.
3. Enforce key-based SSH access and disable direct root login and password authentication.
4. Configure AWS Security Groups and the Linux host firewall as complementary control layers.
5. Apply a default-deny inbound policy and permit only explicitly required services.
6. Remove or disable unnecessary packages, services, and exposed ports.
7. Use Fail2Ban to temporarily block repeated failed SSH authentication attempts.
8. Configure auditd to record sudo usage, account changes, and changes to selected sensitive files.
9. Analyze the systemd journal, authentication logs, and firewall events.
10. Write a Bash security report summarizing failed logins, blocked addresses, disk usage, and service health.
11. Create an EBS snapshot and a file-level backup, then test the documented restore procedure.
12. Generate controlled events, such as failed SSH attempts and unauthorized file changes, and investigate them through logs.

Technology Stack

TECHNOLOGY STACK

Ubuntu Server, Amazon EC2, AWS Security Groups, OpenSSH, Nginx, nftables or UFW, Fail2Ban, auditd, systemd journal, rsyslog, Bash, and Amazon EBS snapshots.

GitHub Deliverables

1. System and network architecture diagram.
2. Installation and hardening guide.
3. Firewall and SSH configurations.
4. Bash security-reporting script.
5. Sanitized log examples.
6. Security-control checklist.
7. Backup and restore runbook.
8. Controlled incident-investigation report.
9. Known limitations and improvement recommendations.

Completion Criteria

Only required ports are exposed, password-based SSH access is disabled, security-relevant events are logged, the Bash report runs successfully, and restoration from the selected backup method is verified.

Scope and Realism

This is a single-server security laboratory. It will not be presented as highly available, interruption-free, or universally production-ready infrastructure.

Project 2 - Secure Containerized Application Platform with Infrastructure as Code

> Timeline: After completing Level 2.

Objective

Use web, API, database, Docker, Kubernetes, Terraform, and Ansible knowledge to create a secure, repeatable, and tangible application platform.

What Will Be Built?

A small three-tier application consisting of an Nginx entry layer, a compact REST API, and a PostgreSQL database.

The application will run locally with Docker Compose before being deployed to a single-node k3s or equivalent lightweight Kubernetes environment on one AWS EC2 instance or local virtual machine prepared with Terraform and Ansible.

Business logic will remain intentionally small. The project focuses on container security, Kubernetes resources, data persistence, and repeatable infrastructure rather than application complexity.

Implementation Work

1. Separate the API, database, and reverse proxy into individual containers.
2. Use multi-stage Docker builds.
3. Run containers as non-root users where supported.
4. Reduce image size and remove unnecessary packages.
5. Keep sensitive values out of images and Git repositories.
6. Create a local development and validation environment with Docker Compose.
7. Define network rules, the EC2 instance, and supporting AWS resources with Terraform.
8. Automate operating-system preparation, the container runtime, and k3s installation with Ansible.
9. Create Kubernetes Deployments, Services, Ingress, ConfigMaps, Secrets, and persistent-storage resources.
10. Add startup, readiness, and liveness probes.
11. Define CPU and memory requests and limits.
12. Reduce unnecessary privileges through RBAC.
13. Restrict API-to-database traffic with NetworkPolicy.
14. Perform a PostgreSQL backup and restore test.
15. Destroy and rebuild the environment using Terraform and Ansible.

Technology Stack

> **TECHNOLOGY STACK**
>
> Docker, Docker Compose, Nginx, a compact REST API, PostgreSQL, Kubernetes or k3s, Terraform, Ansible, Amazon EC2, Amazon S3 remote state, and a selected CNI implementation.

GitHub Deliverables

1. Application and infrastructure architecture.
2. Dockerfiles and Compose definitions.
3. Terraform modules.
4. Ansible playbooks and role structure.
5. Kubernetes manifests.
6. RBAC and NetworkPolicy explanations.
7. Database backup and restore procedure.
8. Security-scanning results.
9. Installation, removal, and rebuild documentation.
10. Known limitations.

Completion Criteria

> The application can be built from source, containers run with secure baseline settings, data survives restarts, NetworkPolicy tests produce the expected result, and the environment can be recreated without undocumented AWS Console steps.

Scope and Realism

> A single-node Kubernetes environment is acceptable for cost and learning objectives. It will not be described as a highly available Kubernetes cluster.

Project 3 - Secure CI/CD, GitOps and Observability Platform

> Timeline: After completing Level 3.

Objective

Build a DevSecOps platform that tests the Level 2 application, enforces security checks, produces container images, deploys through GitOps, and observes the system through metrics, logs, and traces.

What Will Be Built?

The application from Project 2 will continue to be used. Instead of creating another application, this milestone adds software-delivery, security, observability, and operational layers.

Changes will pass through pull-request controls. A successful pipeline will publish a versioned image to a registry, update the deployment repository, and allow Argo CD to reconcile the staging environment.

Implementation Work

1. Configure GitHub branch protection and a pull-request workflow.
2. Add unit tests, linting, and basic integration tests to the pipeline.
3. Perform secret scanning with Gitleaks.
4. Scan dependencies and container images.
5. Scan images and Kubernetes manifests with Trivy.
6. Generate an SBOM.
7. Publish successful builds to GHCR or Amazon ECR.
8. Use commit SHAs or meaningful version tags instead of latest.
9. Separate the application repository from the deployment repository.
10. Deploy declaratively to a staging environment with Argo CD.
11. Collect application and system metrics with Prometheus.
12. Create Grafana dashboards.
13. Centralize application and platform logs with Loki.
14. Produce at least one end-to-end request trace with OpenTelemetry.
15. Create alerts for service unavailability and elevated error rate with Alertmanager.
16. Write Bash or Python smoke-test and deployment-validation scripts.
17. Test broken images, failed health checks, and high-error-rate scenarios.
18. Perform a controlled rollback and write a concise blameless postmortem.

Technology Stack

TECHNOLOGY STACK

GitHub Actions, GitHub Container Registry or Amazon ECR, Gitleaks, Trivy, SBOM tooling, Argo CD, Kubernetes or k3s, Prometheus, Grafana, Loki, OpenTelemetry, Alertmanager, Bash, and Python.

GitHub Deliverables

1. CI/CD workflow definitions.
2. Security-control results.
3. GitOps repository structure.
4. Argo CD deployment flow.
5. Grafana dashboard screenshots and JSON definitions.
6. Prometheus alert rules.
7. Log and trace examples.
8. Smoke-test scripts.
9. Evidence of a failed deployment and successful rollback.
10. Incident timeline and blameless postmortem.

Completion Criteria

A commit that fails a required security gate cannot be deployed; a clean commit automatically produces an image and reaches staging; dashboards display telemetry; defined alerts can be triggered; and a faulty release can be rolled back to the previous working version.

Scope and Realism

A single staging environment is sufficient at this level. The project will not claim to be a full-scale enterprise CI/CD platform or a real production environment.

Project 4 - Resilient AWS Service Platform and SRE Validation

> Timeline: After completing Level 4.

Objective

Move the application and delivery workflow from earlier projects to a more resilient, measurable, and recoverable AWS service architecture, then validate SRE, performance, and disaster-recovery principles.

What Will Be Built?

A cost-controlled AWS architecture spanning two Availability Zones and provisioned with Terraform.

The baseline architecture includes a dedicated VPC, subnets across two Availability Zones, a public Application Load Balancer, two small application instances or an Auto Scaling Group in private subnets, images pulled from Amazon ECR, an RDS PostgreSQL database or documented cost-conscious alternative, CloudWatch telemetry, and backup and restore mechanisms.

The environment will exist only for controlled tests and will be destroyed with Terraform after validation to avoid unnecessary cost.

Implementation Work

1. Provision the VPC, subnets, route tables, Internet Gateway, Security Groups, and IAM roles with Terraform.
2. Distribute the application across two Availability Zones.
3. Configure ALB health checks and target groups.
4. Evaluate AWS Systems Manager access instead of exposing direct public SSH.
5. Pull the application image from Amazon ECR.
6. Manage sensitive values with Secrets Manager or Parameter Store.
7. Send application and system logs to CloudWatch.
8. Define basic SLIs for availability, latency, and error rate.
9. Define a realistic but limited SLO.
10. Perform load testing with k6 or an equivalent tool.
11. Terminate one application instance deliberately and measure ALB and Auto Scaling behavior.
12. Generate alarms for high CPU, elevated error rate, or service loss.
13. Back up the database or application state and perform a restore test.
14. Define laboratory-appropriate RTO and RPO targets.
15. Produce an incident timeline, runbook, and postmortem after failure tests.
16. Add an AWS Budget and a documented cost estimate.
17. Destroy and recreate the complete environment with Terraform.

Technology Stack

TECHNOLOGY STACK

AWS VPC, EC2, Auto Scaling Group, Application Load Balancer, Amazon ECR, IAM, Systems Manager, RDS PostgreSQL or a documented alternative, S3, CloudWatch, AWS Secrets Manager or Parameter Store, Terraform, GitHub Actions, and k6.

GitHub Deliverables

1. AWS architecture diagram.
2. Terraform modules.
3. IAM and network-security explanation.
4. SLI, SLO, and error-budget document.
5. CloudWatch dashboard and alarm configuration.
6. Load-test results.
7. Failure-test results.
8. Backup and restore report.
9. Incident runbook and postmortem.
10. Cost estimate and shutdown procedure.
11. Architectural limitations and additional improvements required for production.

Completion Criteria

> Loss of one application instance does not cause a prolonged outage, health checks and alarms work as expected, load-test results are documented, data restoration is verified, and the complete environment can be recreated and removed with Terraform.

Scope and Realism

> This is a production-oriented laboratory, not a real production system. Multi-Region deployment, a fully highly available database, and continuously running expensive services are optional architectural extensions rather than mandatory claims.

Project 5 - Secure RAG Security Operations Assistant

> Timeline: After completing Level 5.

Objective

Develop a secure AI application that combines AI security, RAG, LLM application security, data protection, MLSecOps, red teaming, monitoring, and governance.

What Will Be Built?

A compact RAG-based Security Operations Assistant that works with the user's Learning Journal documents, runbooks, and project notes.

The application will retrieve information only from authorized documents, cite the sources used in its answers, and will not execute commands directly on systems in its first version. The goal is to demonstrate secure AI application architecture and controls, not to create an autonomous hacking agent.

Implementation Work

1. Prepare Learning Journal documents as a controlled source set.
2. Split documents into chunks and label them with metadata.
3. Build the embedding and vector-search layer.
4. Add user authentication.
5. Enforce document- or collection-level authorization.
6. Ensure users receive results only from sources they are authorized to access.
7. Cite retrieved sources in model answers.
8. Create direct prompt-injection tests.
9. Test indirect prompt injection with documents containing malicious instructions.
10. Treat retrieved content as untrusted data.
11. Add controls that reduce movement of sensitive data and secrets into prompts, logs, and responses.
12. Apply input and output validation.
13. Add rate limits, token limits, and spending limits.
14. Record prompt, retrieval, model-response, and security-event telemetry.
15. Avoid retaining unnecessary personal or sensitive data in audit logs.
16. Produce a threat model and data-flow diagram.
17. Map test cases to OWASP LLM risks and MITRE ATLAS techniques.
18. Create repeatable automated security-evaluation tests.
19. Report red-team results, successful controls, bypasses, and unresolved weaknesses.
20. Produce a model card, system card, AI risk register, and acceptable-use boundaries.
21. Perform an incident-response exercise for a poisoned document or unauthorized-retrieval scenario.
22. Package the application with Docker and scan and deploy it through the pipeline created in earlier projects.

Technology Stack

TECHNOLOGY STACK

Python, FastAPI, an LLM API or local model, an embedding model, PostgreSQL with pgvector or Qdrant, Docker, an authentication mechanism, Prometheus-compatible metrics, centralized logging, GitHub Actions, Trivy, Gitleaks, and foundational AI-security evaluation tools.

GitHub Deliverables

1. AI application and RAG data-flow diagram.
2. Threat model.
3. Authentication and authorization design.
4. Document-ingestion pipeline.
5. Prompt-injection test set.
6. RAG-poisoning and unauthorized-retrieval tests.
7. Security-evaluation results.
8. Audit and monitoring examples.
9. Model card and system card.
10. AI risk register.
11. Red-team report.
12. Incident-response report.
13. Secure-deployment documentation.
14. Known limitations and residual risks.

Completion Criteria

Unauthorized users cannot retrieve protected documents, answers cite their sources, prompt-injection tests are recorded, controls are validated through repeatable tests, unnecessary sensitive data is absent from logs, and the application can be deployed reproducibly as a container.

Scope and Realism

This is not a model-training project. It demonstrates secure AI application architecture, development lifecycle, testing, monitoring, and governance. The project will not claim that its controls prevent every attack; test coverage, failures, bypasses, and residual risks will be documented explicitly.

Five-Project Progression Summary

LEVEL 1	A hardened, monitored, and auditable Linux server.
LEVEL 2	A repeatable secure container and Kubernetes application platform.
LEVEL 3	A DevSecOps delivery platform with security gates, GitOps, and observability.
LEVEL 4	An AWS service architecture validated for resilience, performance, SLOs, and recovery.
LEVEL 5	A secure AI application with identity, authorization, RAG security, red teaming, monitoring, and governance controls.

Documentation Approach

Each roadmap topic may include, depending on its requirements:

- English technical documentation
- Architecture and data-flow diagrams
- Security risks and defensive controls
- Hardening recommendations and their limitations
- Command and configuration examples
- Troubleshooting methodology
- Practical validation steps
- Production and laboratory environment distinctions
- Official references and standards

The goal is not to create a separate large laboratory project for every topic.

Small exercises are included when they improve understanding. Broader implementations are consolidated into milestone portfolio projects so that related technologies can be demonstrated together within realistic system boundaries.